Pimpri-Chinchwad Education Trust's
Pimpri-Chinchwad College of Engineering
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU) ISO 21001:2018 Certified by TUV

DEPARTMENT OF INFORMATION TECHNOLOGY

Assignment No. 4

Subject	DBMS Lab
Assignment No.	4
Topic	Implement Procedure and Function.
Student Name	Madhav Khobare
Roll Number	124B1F023
Date	10/03/2026
Div & Batch	SY IT A1
Guided by: -	Ms. Alpana A. Borse

Academic Year: 2025-26 Semester: IV

Aim:

To study and implement stored procedures and functions in SQL for efficient database operations.

Objectives:

- To understand the concept of stored procedures and functions.
- To create and execute procedures in SQL.
- To implement functions with and without parameters.
- To improve reusability and modularity in database operations.

Theory: Stored Procedures and Functions

Stored Procedures are precompiled SQL statements that are stored in the database and can be executed multiple times. They help in reducing code redundancy and improving performance. Procedures can accept input parameters, return output parameters, and include control structures like loops and conditions.

Functions are similar to procedures but always return a value. They are mainly used for calculations and can be used within SQL statements like SELECT. Functions improve modularity and make queries more efficient and reusable.

Both procedures and functions are important for managing complex database operations and enhancing the overall performance of database applications.

Case Study: Airline Management System

SQL Queries (Procedure & Function):

Part A: Stored Procedures

---

1. Create Procedure:

Procedure to display all flights.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE GetAllFlights()
   -> BEGIN
   ->     SELECT * FROM flight;
   -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

---

2. Call Procedure

```
mysql> CALL GetAllFlights();
+-----------+-------------+--------+-------------+--------+------------+
| flight_id | flight_name | source | destination | price  | airline_id |
+-----------+-------------+--------+-------------+--------+------------+
| 101       | AI101       | Mumbai | Delhi       | 5500   | 1          |
| 102       | IN202       | Pune   | Bangalore   | 4500   | 2          |
| 103       | EM404       | Dubai  | Mumbai      | 15000  | 3          |
+-----------+-------------+--------+-------------+--------+------------+
3 rows in set (0.00 sec)
```

---

3. Procedure with Parameter:

Get flights by source city.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE GetFlightsBySource(IN src VARCHAR(50))
   -> BEGIN
   ->     SELECT * FROM flight WHERE source = src;
   -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

---

4. Call Procedure with Parameter

```
mysql> CALL GetFlightsBySource('Mumbai');
+-----------+-------------+--------+-------------+--------+------------+
| flight_id | flight_name | source | destination | price  | airline_id |
+-----------+-------------+--------+-------------+--------+------------+
| 101       | AI101       | Mumbai | Delhi       | 5500   | 1          |
+-----------+-------------+--------+-------------+--------+------------+
1 row in set (0.00 sec)
```

---

5. Procedure with OUT Parameter:

Count total passengers.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE CountPassengers(OUT total INT)
    -> BEGIN
    ->     SELECT COUNT(*) INTO total FROM passenger;
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

---

6. Call OUT Procedure

```
mysql> CALL CountPassengers(@total);
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT @total;
+--------+
| @total |
+--------+
| 3      |
+--------+
1 row in set (0.00 sec)
```

Part B: Functions

---

7. Create Function:

Function to get total number of flights.

```
mysql> DELIMITER //
mysql> CREATE FUNCTION TotalFlights()
    -> RETURNS INT
    -> DETERMINISTIC
    -> BEGIN
    ->     DECLARE total INT;
    ->     SELECT COUNT(*) INTO total FROM flight;
    ->     RETURN total;
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

8. Call Function

```
mysql> SELECT TotalFlights();
+----------------+
| TotalFlights() |
+----------------+
| 3              |
+----------------+
1 row in set (0.00 sec)
```

---

9. Function with Parameter:

Get price of a specific flight.

```
mysql> DELIMITER //
mysql> CREATE FUNCTION GetFlightPrice(fid INT)
    -> RETURNS DECIMAL(10,2)
    -> DETERMINISTIC
    -> BEGIN
    ->     DECLARE fprice DECIMAL(10,2);
    ->     SELECT price INTO fprice FROM flight WHERE flight_id = fid;
    ->     RETURN fprice;
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

---

10. Call Function with Parameter

```
mysql> SELECT GetFlightPrice(103);
+----------------------+
| GetFlightPrice(103)  |
+----------------------+
| 15000.00             |
+----------------------+
1 row in set (0.00 sec)
```

Case Study: Civic Voice Interface System

SQL Queries (Procedure & Function):

Part A: Stored Procedures

---

1. Create Procedure:

Display all users.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE GetAllUsers()
   -> BEGIN
   ->     SELECT * FROM user;
   -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

---

2. Call Procedure

```
mysql> CALL GetAllUsers();
+---------+--------------+------------------------+--------------+----------+
| user_id | name         | email                  | phone_number | language |
+---------+--------------+------------------------+--------------+----------+
| 1       | Rahul Sharma | rahul@email.com        | 9876543210   | English  |
| 2       | Neha Verma   | neha@email.com         | 8765432109   | Hindi    |
| 3       | Amit Patil   | amit@email.com         | 7654321098   | Marathi  |
+---------+--------------+------------------------+--------------+----------+
3 rows in set (0.00 sec)
```

---

3. Procedure with Parameter:

Get complaints by status.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE GetComplaintsByStatus(IN stat VARCHAR(20))
   -> BEGIN
   ->     SELECT * FROM complaint WHERE status = stat;
   -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

---

4. Call Procedure

```
mysql> CALL GetComplaintsByStatus('Pending');
+--------------+---------+----------------+---------+
| complaint_id | user_id | issue          | status  |
+--------------+---------+----------------+---------+
| 1            | 1       | Road damage    | Pending |
| 3            | 3       | Garbage issue  | Pending |
+--------------+---------+----------------+---------+
2 rows in set (0.00 sec)
```

---

5. Procedure with OUT Parameter:

Count total complaints.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE CountComplaints(OUT total INT)
    -> BEGIN
    ->     SELECT COUNT(*) INTO total FROM complaint;
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

---

6. Call OUT Procedure

```
mysql> CALL CountComplaints(@total);
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT @total;
+--------+
| @total |
+--------+
| 3      |
+--------+
1 row in set (0.00 sec)
```

Part B: Functions

---

7. Create Function:
Count total users.

```
mysql> DELIMITER //
mysql> CREATE FUNCTION TotalUsers()
    -> RETURNS INT
    -> DETERMINISTIC
    -> BEGIN
    ->     DECLARE total INT;
    ->     SELECT COUNT(*) INTO total FROM user;
    ->     RETURN total;
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

---

8. Call Function

```
mysql> SELECT TotalUsers();
+--------------+
| TotalUsers() |
+--------------+
| 3            |
+--------------+
1 row in set (0.00 sec)
```

---

9. Function with Parameter:

Get complaint status by ID.

```
mysql> DELIMITER //
mysql> CREATE FUNCTION GetComplaintStatus(cid INT)
    -> RETURNS VARCHAR(20)
    -> DETERMINISTIC
    -> BEGIN
    ->     DECLARE stat VARCHAR(20);
    ->     SELECT status INTO stat FROM complaint WHERE complaint_id = cid;
    ->     RETURN stat;
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> DELIMITER ;
```

---

10. Call Function

```
mysql> SELECT GetComplaintStatus(2);
+-------------------------+
| GetComplaintStatus(2)   |
+-------------------------+
| Resolved                |
+-------------------------+
1 row in set (0.00 sec)
```

Conclusion:

In this assignment, stored procedures and functions were successfully implemented for performing database operations. These concepts help in improving code reusability, reducing redundancy, and enhancing database performance. Hence, procedures and functions are essential tools in SQL-based applications.